Solving optimisation problems comparing classical and quantum inspired methods

Introduction

Many real-world engineering problems require selecting the best option from an enormous number of possibilities, with applications in logistics, finance, communication networks, and machine learning. These **combinatorial optimisation** problems become increasingly difficult to solve efficiently as size grows, because the number of possible solutions rises *exponentially*. This makes it important to compare solver performance across several problem sizes. A widely used benchmark for this is the Max-Cut problem, which is suitable for **classical, quantum-inspired, and exact methods**.

Problem Overview

The **Max-Cut problem** seeks to partition the vertices of some undirected graph $G(V,E)$ into two sets such that the number of edges between them, considered *cut* is as large as possible. Although easy to state and visualise, it is **computationally difficult** to solve exactly for large, dense graphs. This makes it a useful **benchmark** for comparing optimisation methods. Max-Cut was used to compare, simulated annealing, simulated quantum annealing, Google OR's CP-SAT, and Gurobi. The primary focus is on how **solver performance** changes with problem size and density.

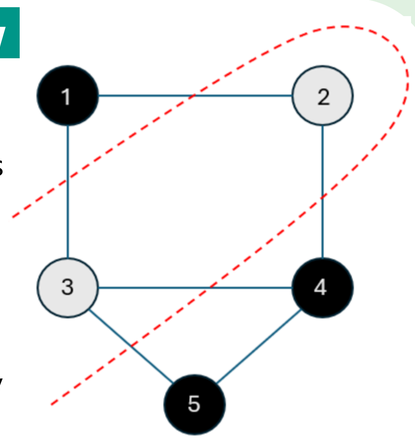


Fig.1: *Example of a Max-Cut solution on a simple graph*

Methods

1 Dataset

Unweighted **Erdős–Rényi graphs** were generated to benchmark solver performance while controlling for **problem size** and **graph density**. The dataset covers graphs from 100 to 1,000,000 nodes and expected degree values from 0.6 to 20, and 10 instances per class. Expected degree is the *average* number of neighbours each node has, so larger values mean denser graphs.

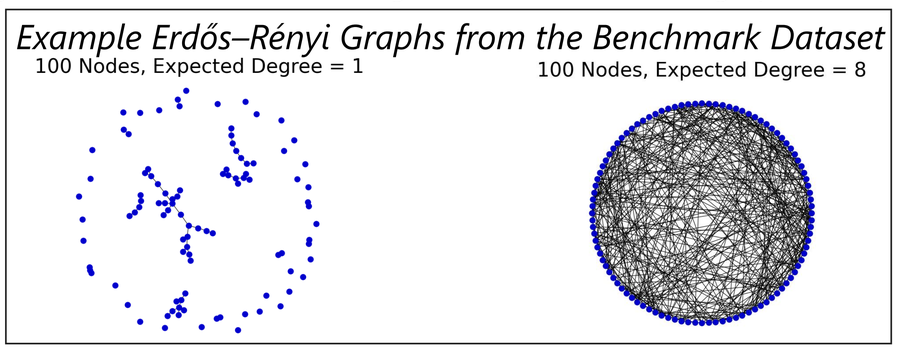
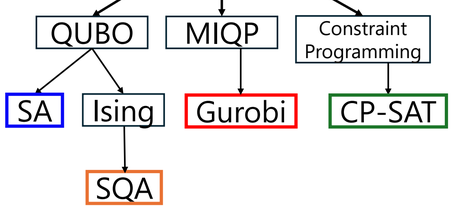


Fig.2: *Example Erdős–Rényi graphs from the dataset. left, sparse instance 100_1.0_0.txt. right, dense instance 100_8_0.*txt

2 Problem Encoding

To compare methods fairly, the same **Max-Cut** objective was written in solver appropriate form. Binary variables $x_i \in \{0,1\}$ represent the two sets of vertices, with edges contributing when their endpoints differ. SA and SQA use QUBO, Gurobi uses MIQP, and CP-SAT uses constraint programming.

Objective: $$max \sum_{(i,j)\in E} x_i + x_j - 2x_i x_j$$



3 Solvers

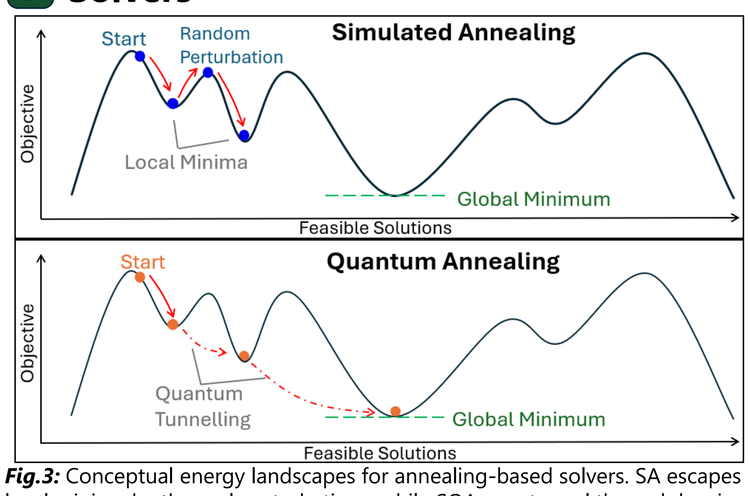


Fig.3: Conceptual energy landscapes for annealing-based solvers. SA escapes local minima by thermal perturbations, while SQA may tunnel through barriers

Four solvers were benchmarked: **simulated annealing** (SA), **simulated quantum annealing** (SQA), Google OR-Tools **CP-SAT**, and **Gurobi**. SA and SQA were included as heuristic methods for finding high-quality approximate solutions, while CP-SAT and Gurobi provided open-source and commercial mathematical programming standards, respectively.

4 Assessment Metrics

Performance was assessed using **objective value** and **runtime**. Objective value measures solution quality, while runtime measures the computational cost. Raw objective value increases naturally with graph size and density, solver quality was compared **relative to the best solution** found within each instance class.

$$Relative\ performance = \frac{solver's\ objective}{best\ objective\ in\ class}$$

Results

1 Performance on Mid-Sized Problems

The first two plots show **objective value** and **runtime** for 10,000 node instances as expected degree increases. Objective value rises with graph density, as denser graphs contain more edges that can be cut. Runtime shows more **method dependent behaviour**, illustrating differences in scalability.

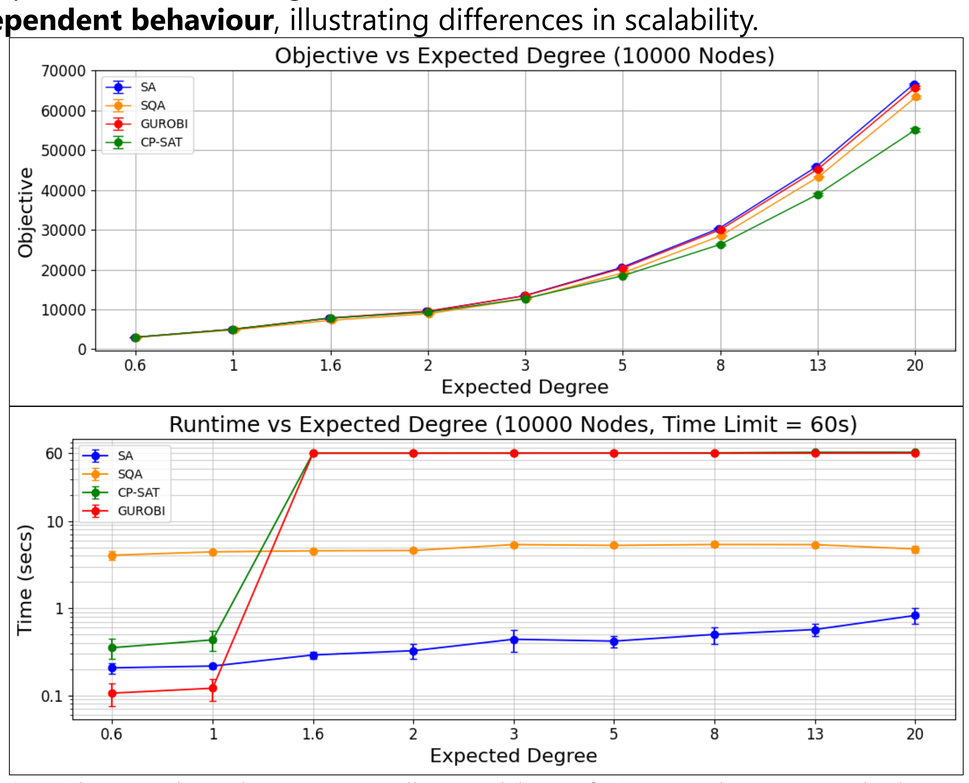


Fig.4: *Objective value and runtime versus all expected degrees for 10,000 node instances in the dataset.*

2 Solver Performance Across Different Regimes

The heatmaps show how **solver performance** *changes* across problem size and density. By splitting sparse and dense regimes at 1,000 and 100,000 nodes. These highlight differences that are less visible in the above plots and makes it easier to compare behaviour across problem regimes.

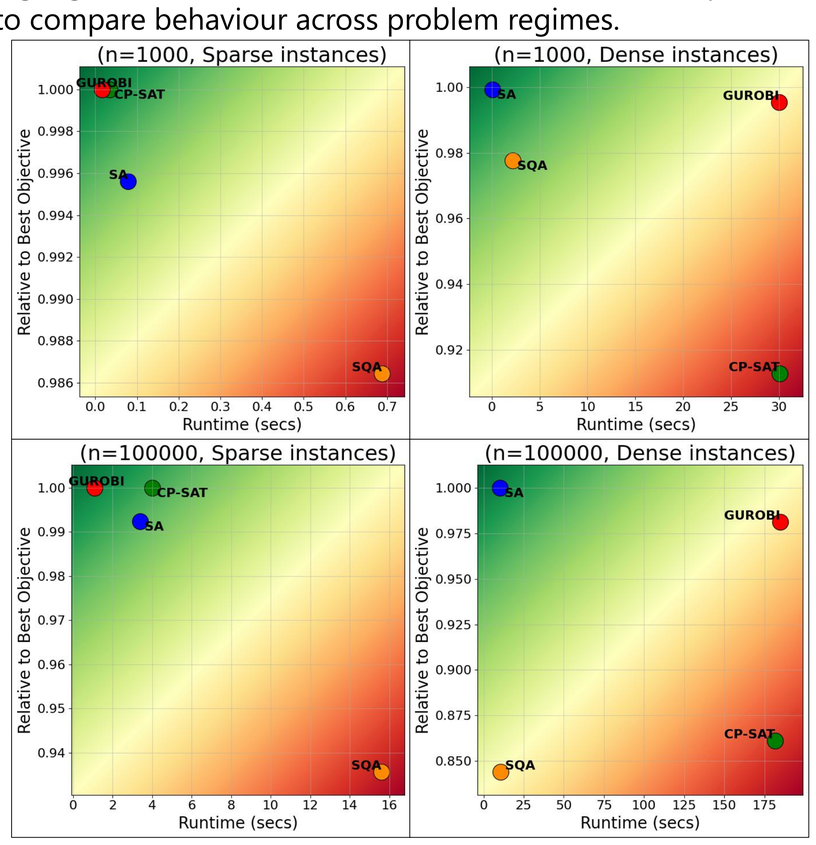


Fig.5: *Heatmaps of solver performance for sparse* (expected degree ≤ 1) *and dense* (expected degree ≥ 2) *graph classes at 1,000 and 100,000 nodes.*

Conclusion

The above results show that solver performance is strongly **regime dependent**. Simulated annealing was the strongest overall method, while simulated quantum annealing became more competitive on denser graphs. CP-SAT and Gurobi performed well on sparse instances, but runtime scaled less effectively as problem size increased.

Practical Interpretation

The best solver depends on the application. In areas such as **finance** or logistics, it may be worth it to accept long runtimes to get the **best objective**. However, for applications such as **network routing** or real time decision making, a **good solution found quickly** may be more valuable than a slightly better solution, obtained slower.

Limitations

The main limitation is that the benchmark uses **unweighted Erdős–Rényi graphs**, which are simple and artificial random graphs and the technique for generating these graphs often produces very uniform instances. This means the dataset instances may not reflect the structure of many **real-world** problems.

Lachlan Walker

ACKNOWLEDGEMENTS: Special thanks to Elena Blokhina for their supervision and advice throughout this project and to the Gurobi team and Academic / WLS Licence Support for providing access to Gurobi for academic use.

UCD DUBLIN